Too Good to Be True? Helpfulness and Deception Online

Erasmus School of Economics

Master Thesis Data Science and Marketing Analytics

October 1, 2025

Author: Anastasis Gkoutas
Supervisor: dr. Michel van de Velden
Second Assessor: dr. Vardan Avagyan

ABSTRACT

Online reviews influence everyday decisions, yet not all of them are genuine and not all of them are equally useful. This thesis investigates two connected problems: the detection of fake reviews and the prediction of helpfulness. Using real-world Yelp restaurant data we address these problems by applying three machine learning methods: LASSO regression, Random Forest, and XGBoost. XGBoost delivers the strongest overall performance, with review depth and readability being the most important features. In the Random Forest model, review depth and rating are the most influential, whereas in the LASSO model, more weight is placed on specific words with strong associations. Helpfulness is easier to predict than deception, because longer and more readable reviews are consistently rated as more useful across the three models. Detecting fake reviews is less clear, because fake reviews closely resemble genuine ones. Looking at feature importance, the results show that vocabulary alone is not enough. The style, clarity, and detail of a review plays is far more important in shaping perceptions. Taken together, the findings reveal that while deception remains hard to capture, the ingredients of helpfulness are much more straightforward, resting on readability and depth rather than authenticity itself.

TABLE OF CONTENTS

1. INTRODUCTION **4**

2. LITERATURE REVIEW **6**

2.1 Online Reviews 6

2.2 Fake Online Reviews 6

2.3 Review Helpfulness 7

2.4 Helpfulness in fake vs. real reviews 8

3. DATA **10**

3.1 Description of the Data 10

3.1.1 Polarity 11

3.1.2 Subjectivity 11

3.1.3 Readability 12

3.1.4 Review Depth 13

3.2 Data Preparation 14

4. METHODOLOGY **16**

4.1 Term Frequency-Inverse Document Frequency (TF-IDF) 16

4.2 Logistic Regression 17

4.3 Random Forest 18

4.4 Boosting 20

4.5 Model Evaluation Metrics 22

5. RESULTS **23**

5.1 Hyperparameter Tuning of the Models 23

5.2 Model Performance Comparison 25

5.3 Feature Importance Analysis 27

6. CONCLUSION & DISCUSSION **31**

7. REFERENCES **32**

8. APPENDIX **37**

1. INTRODUCTION

In today's digital era, customers usually rely heavily on online reviews when considering a purchase. These reviews impact the consumer purchase intentions, either positively or negatively, shaping the final decision. This is why customer reviews are so important for the success of brands and their products. Companies spend a lot of money on getting good word-of-mouth to build a strong market presence and improve their reputation. Some businesses do this to change online reviews to make their products look better and change how customers feel about them. For example, in 2012, the UK Advertising Standards discovered that TripAdvisor had been responsible for not verifying fake reviews, with about 50 million online reviews on its platform without verification of authenticity (Telegraph Reporter, 2012). Similarly, Amazon hosts millions of reviews from many businesses aiming to promote their products. Given the highly competitive environment, it is easy for these businesses to post favorable reviews to increase sales. Indeed, businesses with lower overall ratings are more likely to write fake reviews to make their reputation look better. This study looks into how well machine learning models can find fake reviews and how helpfulness affects deceptive content. This is especially important for users who rely on reviews to make informed choices and for companies whose reputations and sales depend on consumers' trust in review platforms. We examine how polarity, subjectivity, readability, and review depth affect how consumers think about reviews, such as how they rate products, how helpful reviews are, and how trustworthy review platforms are overall. This paper uses a real-world dataset from Yelp, and tries to explain how to recognize misleading and useful content by looking at language patterns.

Among the several attributes linked to online product reviews, 'review helpfulness' is especially significant, as it reflects the subjective assessment of the review as determined by others (Cao et al., 2011). Helpfulness evaluations are a central feature of many large e-commerce platforms, such as Amazon.com, where users can rate reviews based on whether they found them useful, often with prompts like "26 of 32 people found the following review helpful". These helpfulness votes are different from product star ratings and give a different kind of feedback on how good the reviews are. These kinds of systems help other users make better choices, which leads to more satisfaction (Kohli et al., 2004). However, the factors that affect how helpful something is are complicated and not fully understood. Early studies

mostly looked at numbers, such as star ratings, thumbs up/down, or the length of a review (Otterbacher, 2009; Pang et al., 2002). Other studies examine features that make a helpful review, including review polarity, subjectivity, readability, and depth. Polarity captures the sentiment of a review, which is mostly classified into positive, negative, or neutral (Kuan et al., 2015). Subjectivity is the level to which reviews include personal feelings and thoughts (Banerjee et al., 2017). Readability measures how easily understandable the review text is (Ghose and Ipeirotis, 2011), while depth refers to the amount of content an online review provides (Chua and Banerjee, 2015). Researchers have examined these features separately, but they have not examined how well these features work together.

This study's research is a two-step approach. In the first step, different machine learning methods, such as Logistic Regression, Random Forests, and Boosting, are used to predict two outcomes on the same dataset based on textual features of reviews: (1) if the review is seen as helpful, and (2) if it is likely to be fake. Sentiment polarity, subjectivity, readability, and review depth are the four key textual features used, along with other text features. The next step is to look at which are the most important features that make a review helpful, and which features make the review real or fake. The main question of this study is: How do the textual features of fake reviews (polarity, subjectivity, readability, depth) differ from authentic reviews, and which features best explain their perceived helpfulness? Answering this question is important for academic research because it adds to the literature on text analytics by showing how linguistic and structural features can be used to study deception and review impact. It is also relevant for businesses, because companies and review platforms can use these insights to detect manipulation more effectively and strengthen consumer trust in online reviews.

To operationalize our concepts, fake reviews are defined as those flagged as deceptive by Yelp's internal review filter system, indicated by a flagged label in the dataset. Reviews not flagged are considered authentic. Review helpfulness is measured using a score that combines several engagement metrics. "Useful" votes are assigned the most weight, while "compliments," "cool," and "funny" votes contribute to the score with less weight. This produces a continuous measure of how helpful a review is perceived to be by users. For the purposes of modeling, reviews that are in the top 30 percent of this distribution are labeled "helpful," and the rest are labeled "not helpful". This approach ensures that very high numbers of helpful votes are distinguished from cases with only a single vote, capturing more

meaningful differences in perceived helpfulness. Sentiment polarity, subjectivity, readability, and review depth are extracted from each review. These textual features are used both to predict helpfulness and fakeness, and to compare helpful fake reviews with helpful real reviews. The dataset used in this study contains more than 27,000 Yelp reviews in the dataset for this study. Each review has the review text, a label that says whether it is fake or real, and the number of votes from real users.

2. LITERATURE REVIEW

2.1 Online Reviews

Online reviews have emerged as an important aspect of ecommerce, representing a new form of electronic word-of-mouth (eWOM) communication (Chen and Xie 2008). These reviews allow individuals from all over the world to share their personal experiences, thoughts, and opinions on a range of products and services (Barbado et al., 2019). This is a common occurrence now, as customers can leave reviews on specific retail websites or dedicated review websites. Therefore, the importance of online reviews has grown significantly. They are considered the most accessible and widely used form of eWOM, with the number of consumers reading reviews before making a purchase constantly rising (Sandulescu and Ester, 2015). More than 80% of consumers in the United States indicate they use online reviews before purchasing the product (Smith and Anderson, 2016). As a result, they have a strong impact on consumers' purchase decisions and influence the amount of money spent, affecting 20-50% of online purchase decisions (Mathwick and Mosteller, 2016). Beyond influencing individual decisions, real customer reviews are important in the marketplace since they reflect the quality of the product, provide honest feedback to companies for product improvement, and control seller behavior by pointing out bad products (Plotkina et al., 2018).

2.2 Fake Online Reviews

Online reviews are becoming less useful because there is an increasing number of fake reviews (Salminen et al., 2022). These fake online reviews are also called 'fake reviews', 'deceptive reviews', 'deceptive opinion spam', 'review spam' or 'review fraud', and they are fake reviews that were written to sound real, in order to deceive the reader (Salminen et al., 2022). Businesses write fake positive reviews about their products mostly because it can help them make more money and sell more (Yao et al., 2017). Businesses might also post bad reviews about their competitors to hurt their sales and reputation (Mukherjee et al., 2013). Furthermore, individuals may write fake reviews to get rewards, such as receiving payments that can range from 25 cents or 1$ per review (Yao et al., 2017). These reviews appear authentic, but the individuals writing the reviews do not have actual experience with the product or service (Salminen et al., 2022). People can stay anonymous on the internet, which

makes it easy for anyone to post a review without restriction, making it difficult to notice signs of dishonesty that are present in direct interactions (Heydari et al., 2015).

Unlike truthful opinions, fake reviews are not consistent with real product or service evaluations and can be either positive, negative or neutral in sentiment (Sudhakaran et al., 2016). The high rate of fake online reviews is a serious concern, with studies mentioning that 16% of Yelp reviews were flagged as suspicious by the platform's own algorithm (Luca and Zervas, 2016), while the internal quality control process of a European review site found that up to 25% of reviews were rejected (Munzel, 2016). There have also been some other serious cases, like when Samsung was fined for negative fake reviews against a competitor in 2013 (Bates, 2013), and when Amazon sued thousands of fake reviewers in 2015 (Gani, 2015). Instances like these make consumers become suspicious of online reviews, and makes online marketplaces less trustworthy (Ahmad and Sun, 2018). People do not trust online reviews because of this. They might even ignore helpful reviews that come from real customers who are being honest (Mayzlin et al., 2014).

2.3 Review Helpfulness

To deal with the rise of fake reviews, platforms encourage users to rate how helpful a review is. These helpfulness votes show which reviews others find useful and worth reading. Online review helpfulness is usually defined as the subjective evaluation of a review as judged by other users (Cao et al., 2011). This evaluation is based on how useful the review is thought to be in helping people make decisions regarding which products to buy and how relatively simple it is to buy the product (Mudambi et al., 2010). Yelp, Amazon, and TripAdvisor are some of the biggest e-commerce websites that request from users to rate how helpful their reviews are. The companies then use this feedback to help users sort through a lot of information. According to Cao et al. (2011), helpful reviews can assist a consumer's decision-making process and reduce information overload, which is expected to result in a significant increase in revenue for online marketplaces. Companies often write about helpful reviews since these reviews can contribute to the increase in the site's perception of value and social presence, lead to increased traffic, a greater number of retained users, and a crowd of potential buyers (Mudambi et al., 2010).

At first, helpfulness was measured by straightforward quantitative aspects like star ratings and the number of "thumbs up/down" votes a review had, as well as by how long or short a review was (Qazi et al., 2016). However, the way helpfulness is measured online has shifted to include more qualitative factors. Review polarity, or sentiment, is one of the qualitative factors that makes a review helpful. It shows whether the review is mostly positive, negative, or neutral. This is usually based on star ratings (Li et al., 2014). Positive reviews are the most common and usually get the most helpful votes, followed by mixed reviews, with negative reviews being the least likely to be seen as helpful (Chua and Banerjee, 2015). This is happening because of positivity bias, where consumers look for reviews that support their decision before buying something (Chua and Banerjee, 2015). Another feature of review helpfulness is review depth, which looks at how much detail the reviewer gives in the review text (Chua and Banerjee, 2015). People usually think that reviews with more depth are more helpful because they give more information and context about the products (Chua and Banerjee, 2015). However, the relationship between length and helpfulness is non-linear, which means there is an ideal length (Kuan et al., 2015). Reviews that are too long may discourage involvement, but very short reviews may be too simplistic (Kuan et al., 2015). Another important part of how helpful a review is its subjectivity, which shows how much of a review is based on personal opinion and feeling as opposed to facts or objective information (Ghose and Ipeirotis, 2011). Users usually consider reviews more helpful, when they include both subjective and objective opinions (Ghose and Ipeirotis, 2011). Lastly, readability is how easy it is to understand what a review says (Chua and Banerjee, 2016). Reviews that are easy to read are expected to be more helpful, as easy-reading text can improve comprehension, retention, and reading speed (Chua and Banerjee, 2016). Thus, the following is hypothesized:
H1: Helpful reviews with more depth and readability are more likely to be rated as helpful, whether they are fake or real.

2.4 Helpfulness in fake vs. real reviews

Some research shows that people think fake reviews are more helpful than real ones (Wang et al., 2023). Wang et al. (2023) found that 16% of fake, deleted reviews were rated as helpful, while only 13% of real reviews were. Wang et al. (2023) set up two online experiments to see how different ways of being open about things on platforms might change how useful people find reviews and whether they want to buy the product. People were shown reviews that were

more or less clear and then asked to rate how helpful they thought the reviews were. Consumers found fake reviews not only helpful but more helpful, on average, than other reviews. In a similar way, a study on machine-generated fake reviews found that users thought they were just as helpful as real reviews written by people (Yao et al., 2017). In the study by Yao et al. (2017), participants gave unflagged machine-generated reviews an average usefulness score of 3.15, while real Yelp reviews scored 3.28. This shows that people saw the fake reviews as almost just as helpful as the real ones. They often seem helpful because they use certain features that people mistakenly believe make a review more useful. For example, brand repetition, which is common in fake reviews, increased the likelihood of making a helpful judgement by about 73.74%; inconsistent reviewer history was associated with a 61.27% increase in perceived helpfulness; and suspicious language results in a 140% boost in helpfulness judgments (Wang et al., 2023). These findings show that fake reviews can trick people into thinking they are helpful (Wang et al., 2023). Fake reviews not only make people think they are not helpful, but they also hurt the trust and usefulness of the whole review system (Plotkina et al., 2018). In the end, fake reviews often look helpful to customers and can even seem more helpful than real reviews, which is a problem for both users and platforms that need better ways to find fake reviews and make sure people know about them (Salminen et al., 2022). Thus, the following are hypothesized:

H2: Fake reviews are considered more helpful than real reviews.

H3: Easy-to-read text (readability) and detailed content (depth) best explain why both fake and real reviews are rated as helpful.

3. DATA

3.1 Description of the Data

The dataset used in this study consists of 27,000 Yelp restaurant reviews taken from GitHub's repository called Fake-Review-Detection by Darshan Dagly (Dagly, 2020). The creator of this dataset scraped Yelp publicly available reviews from 2006 to 2012, and most of the reviews are from 2010 to 2012. This is one of the few publicly available datasets that includes both fake review labels, flagged by Yelp and helpfulness metrics of the users, and offers an opportunity to discover the relationship between fake reviews and helpfulness. The original dataset had a lot of variables that will help with the analysis. These include reviewID (unique identifier), reviewerID (user identifier), date (the date the review was posted), yelpJoinDate (reviewer account creation date), flagged (Yelp's fake review indicator), reviewUsefulCount (number of helpfulness votes), reviewContent (full text of the review), and rating (1-5 star score).

From the existing variables, several new features were created. Based on Yelp's flagged labels, the Fakeness variable (1=fake, 0=authentic) had 23% fake reviews and 77% real reviews. This imbalance in fake and real reviews is supported by previous research saying that 15-25% of online reviews may be fake, based on Yelp's filtering algorithm that flags suspicious reviews (Luca and Zervas, 2016).

This study developed a composite helpfulness score by integrating various engagement metrics for the analysis of helpfulness. These were the number of "useful" votes, "compliments," and "funny" votes. "Useful" votes were the most important because they are the most directly related to how helpful something is. This created a continuous measure that displayed how helpful other users thought each review was. To create a binary outcome that could be used in the models, the top 30% of this score distribution was marked as helpful with a value of one. The other 70% were given a value of zero and were not helpful. This definition did not equate a single helpful vote with a large number of votes, but it still showed important differences between reviews. Along with these outcome variables, the original review content and its derived features, like polarity, subjectivity, readability, review depth, and TF-IDF terms, were used to predict the traits of reviews.

3.1.1 Polarity

The sentiment polarity of each review was calculated using the sentimentr package in R (Rinker, 2021). This package uses a sentiment lexicon to give words a positive or neutral value, groups them by sentence and review, and changes the values for words that change their meaning, like negations or amplifiers. The scores ranged from -1.45 (very negative) to +2.79 (very positive). As can be seen in Figure 1, the distribution is approximately normal, with the mean just above zero, indicating that most of the reviews are above the neutral mark.

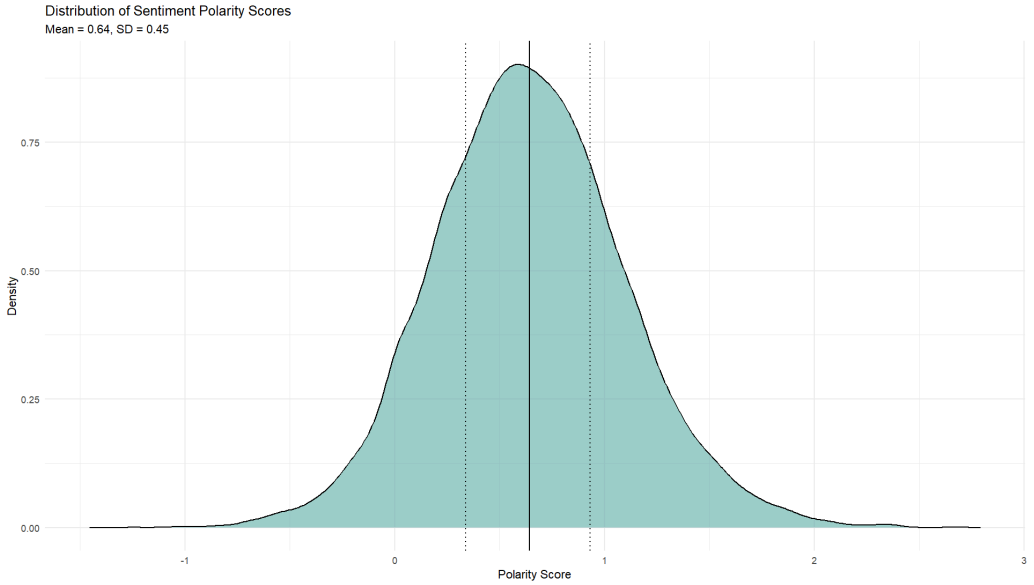

Figure 1: Distribution of Sentiment Polarity

3.1.2 Subjectivity

To measure subjectivity, the Jockers-Rinker lexicon was used, which classifies words as subjective or objective (Rinker, 2019). This makes it possible to figure out how many subjective words are in the text. The scores range from 0 (completely objective) to 0.76 (very subjective). The average subjectivity score was 0.23 (SD = 0.12), meaning that, on average, about 23% of the words in a review were classified as subjective. Previous research suggested that fake reviews might be written in a more factual and objective style in order to avoid suspicion (Heydari et al., 2015). However, as shown in Figure 2, fake reviews in this dataset had slightly higher subjectivity scores than real reviews, although the distributions are largely similar.

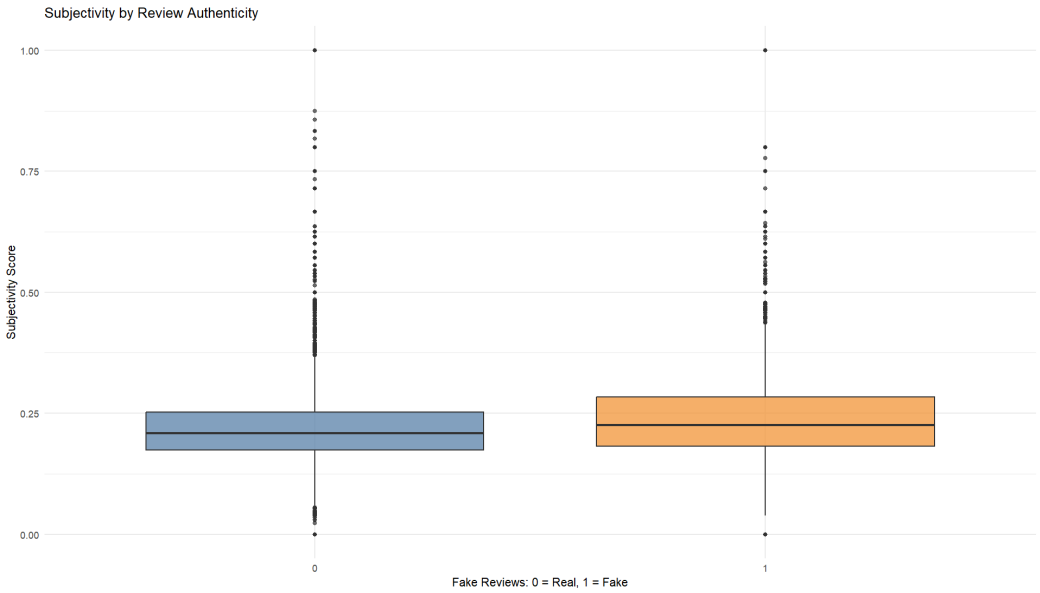

Figure 2: Subjectivity by Review Authenticity

3.1.3 Readability

To measure how easy a review was to read, the Automated Readability Index (aRI) was calculated, which measures how easy each review was to read. This metric is based on the number of characters in each word and the number of words in each sentence, based on U.S. grade levels. The range was from 1 to 14, and most reviews got the highest score of 14 (Mean = 13.33, SD = 1.16). The highest aRI value means that the text is easy to read at the college level. As shown in Figure 3, the distribution is highly uneven, with little variation across reviews. This suggests that the writing style in the dataset is generally simple and accessible, but also that aRI provides limited discriminative power for distinguishing between reviews.

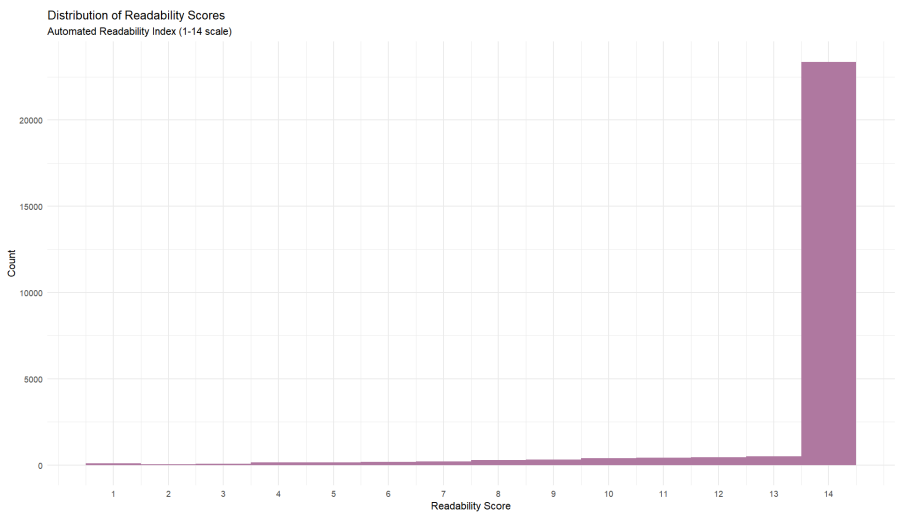

Figure 3: Distribution of Readability Scores

3.1.4 Review Depth

Finally, review depth was measured by the number of words in the reviews. The reviews ranged in length from 1 to 612 words, with an average length of 81.6 words and a median length of 61 words (Figure 4). The histogram showed that the reviews were all different lengths, and many of them were longer than 100 words. This means that the reviews had different amounts of detail and interaction from users.

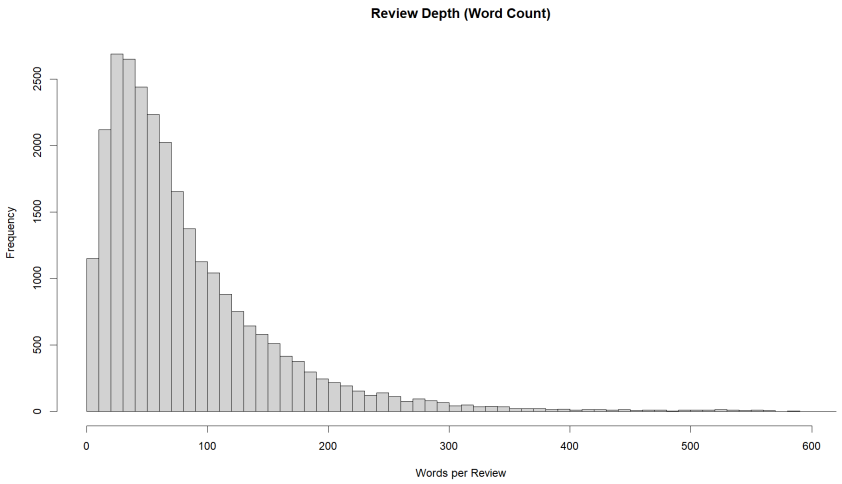

Figure 4: Distribution of Review Depth

3.2 Data Preparation

Data preparation included removing the duplicates and empty or very short reviews with less than 20 characters, in order to make sure that there was enough content for analysis. Also, rows with missing values were removed from the dataset. After these cleaning steps, the final dataset had 26,750 reviews. The different types of features had different scales for measuring them. For example, readability scores went from 1 to 14, word counts went from 1 to 612, and polarity scores went from -1.45 to +2.79. To deal with these differences in scale, word counts were log-transformed to reduce skewness and then standardized, while readability scores were rescaled using range normalization (min-max scaling). The remaining continuous features were standardized. This involved subtracting the mean of each variable and dividing by the standard deviation, meaning that each variable was rescaled to have a mean of zero and a standard deviation of one. This made sure that variables measured on different scales, like word counts and readability scores, have the same effect on the models. Also, the data was split into train and test sets to estimate prediction accuracy. Then, the dataset was split in

two parts, a training set (80%) and a test set (20%). The split was stratified, which means that the ratios of fake and helpful reviews were kept the same in both groups. Using the same split for both tasks (fake detection and helpfulness prediction) made it possible to fairly compare how well the model did on the same data that it had not seen before.

Table 1 shows a summary of the variables that were used in the models. The table sorts the features into five groups: outcome variables, sentiment measures, readability indicators, metadata, and textual features made with TF-IDF. The TF-IDF features show how important 206 terms were in the reviews. These terms were selected by keeping only the top 1000 TF-IDF terms and removing those that appeared in less than 5% of reviews (95% sparsity threshold). The Appendix has the complete list of the TF-IDF features.

Variable group	**Variables**	**Description**
Target variables	Fakeness, Helpfulness	Binary outcomes for fake review detection and helpfulness prediction.
Sentiment features	Polarity, Subjectivity	Polarity measures positive/negative tone. Subjectivity measures opinion vs. fact.
Readability features	Readability, Review Depth	Readability index and total word count.
Metadata	Rating	Original review star rating (1-5).
TF-IDF features	206 variables such as great, pizza, service, delicious	Weighted word frequencies (using TF-IDF) representing the most informative terms in reviews.

Table 1: Overview of variables used in the models

4. METHODOLOGY

This section explains the methods we used to find out how well machine learning algorithms can find fake reviews and predict how helpful reviews will be. The first part of the methodology explains Term Frequency-Inverse Document Frequency (TF-IDF) as the basic way to represent text. Then, it goes into detail about three machine learning algorithms: Logistic Regression, Random Forest, and XGBoost.

4.1 Term Frequency-Inverse Document Frequency (TF-IDF)

Online reviews are written in text, while machine learning methods need numbers to work. TF-IDF is a method that re-weights the raw word counts from the reviews into more informative numerical features. The method examines the importance of each word in the reviews compared to the other reviews in the dataset. Term Frequency (TF) is a way to count how many times a word appears in a review. For example, the word "delicious" has a term frequency of three if it shows up three times in a 100-word review. But just counting words is not enough because some words appear very often across all of the reviews and contribute little to distinguish between different types of reviews. To fix this, Inverse Document Frequency (IDF) looks at how rare each word is in all of the reviews. Words that do not come up very often and are only mentioned in a few reviews get high IDF scores. Words that are used more often get low scores. For example, if "delicious" shows up three times in a review but only 100 times out of 26,750 reviews, it is considered rare and gets a high score. TF-IDF goes up when a word is used more often in a single review (TF) and goes down when it is used a lot in all reviews (IDF). Words that appear often in a certain review but are uncommon in the overall dataset are usually more helpful, because they differentiate between the different kinds of reviews.

The initial step in the TF-IDF method was to make a corpus from all the review texts and preprocess it for analysis. The cleaning process was implemented by converting everything to lowercase, deleting punctuation and numbers, removing stop words such as “and”, “the”, and “is”, and removing extra spaces. A document-term matrix was created where each row represents a review and each column represents a word. Instead of simple word counts, the matrix cells contained TF-IDF scores, which reflect how important a word is in a specific review relative to all other reviews. The initial matrix had thousands of words which would

be too many for the models to process. To reduce this, this study first selected the top 1,000 words with the highest total TF-IDF scores across all reviews, meaning the words that contributed the most overall importance in the dataset. Next, sparsity screening was applied to remove words that appeared in less than 5% of the reviews (i.e., fewer than 1,338 reviews). This final step reduced the matrix to 206 TF-IDF features meaning 206 columns (words) remained in the document-term matrix.

4.2 Logistic Regression

Logistic regression is a simple but robust technique for binary classification that models the probability of an observation belonging to one of two categories. Logistic regression, unlike regular multivariate regression, models the chance of binary outcomes. It does this by assuming that the log-odds of the result have a linear relationship with the input variables. The logistic regression model finds the best coefficients for the input features that can tell the two groups apart. It uses the logistic function to make sure that all of the predictions are between 0 and 1. The review is put into the positive class if the predicted probability is higher than 0.5. This means that it is classified in the positive class, and it is either fake or helpful. It goes into the negative class (real or not helpful) if it is less than 0.5. Each feature gets a coefficient that shows how much it affects the prediction and in what way. If a coefficient is positive, it means that the review is more likely to be fake or helpful if that feature has a higher value. If the coefficient is negative, it means the opposite.

Regularization can be added to logistic regression to make it work in situations where there are many predictors, known as high-dimensional data, and the model risks fitting noise instead of real patterns, which is called overfitting. The Least Absolute Shrinkage and Selection Operator (LASSO) is a popular extension of Logistic regression that adds a penalty to the model based on the absolute size the coefficients are. This penalty makes some coefficients exactly zero, which takes out features that do not add any information to the model. LASSO does both regularization, which lowers the chance of overfitting, and feature selection, which keeps only the most important predictors for the model. A tuning parameter, which is usually chosen through cross-validation, controls how strong the penalty is. A bigger penalty sets more coefficients to zero, which makes the model simpler. A smaller penalty lets

more features remain in the model. This is why LASSO is helpful for text data, where there can be a lot of possible predictors.

The penalty parameter, which is usually called lambda (λ), is the most important hyperparameter in LASSO regression. This setting controls how much the model punishes large coefficients. When lambda is set to zero, LASSO is the same as regular logistic regression with no penalty and all features are included. As lambda goes up, more coefficients get closer to zero, and some may even be set to zero, which takes them out of the model. A model with only a few predictors is very simple when lambda is very large. When lambda is very small, the model is more complex and may overfit. It is important to choose the right value of lambda to find the right balance between generalization and complexity. In practice, cross-validation is used to choose this parameter. Cross-validation is a method that splits the training data into several folds. The model is trained on part of the data and validated on the remaining fold. This process is repeated so that each fold is used once as validation, and the average performance across folds gives a more reliable estimate of how well the model will generalize. The model is trained with different values of lambda, and the one that works best on validation data is chosen. This approach ensures that the chosen lambda generalizes well to new data.

4.3 Random Forest

Random Forest is an ensemble method that combines decision trees to make better predictions. A single decision tree often suffers from high variance because small changes in the training data can lead to very different tree structures. This makes trees prone to overfitting and less reliable on new data. Each decision tree sorts reviews by making a series of yes or no choices about the features. The tree provides a model to make predictions by asking a sequence of yes or no questions that split the data until it ends with a group of similar observations, and that group is then used to make the final prediction. But individual trees can easily overfit to the training data, which means they might do well on the training data but not on new data that are not seen before. Random Forest solves this problem by making a lot of trees and then using majority voting to make the final prediction. The algorithm builds each tree by taking a random sample of the training data with replacement, which is a bootstrap sample. At each decision point in the tree, only a random subset of features is considered for splitting. This randomness in the samples and features increases the

diversity of the trees, making them less likely to all look the same. Without randomness, many trees would look similar because they rely on the same data and strong predictors. Because the predictions are combined, the overall model is usually more accurate and less likely to overfit than a single decision tree. The final prediction for the classification is based on the majority vote of the trees. Random Forest also works well with high-dimensional data, and it does not need feature scaling like logistic regression. This is because they use splitting thresholds on features instead of distance measures. Random Forest also gives feature importance scores, but because they are calculated in a non-linear way, they can be harder to interpret compared to linear models.

There are a few important hyperparameters that affect how well a Random Forest model works. One of the hyperparameters in Random Forest is "ntree," which is the number of trees in the forest. This determines how many decision trees will be created. More trees usually mean more stable and accurate predictions because the model can use more trees to make predictions. But it also takes longer to train the model because more trees mean additional calculations. In practice, adding more trees to the model usually makes the performance of the model better, but after a certain point, the improvements stop. So, the number of trees is set high enough to keep things stable without making the calculations more complicated than they need to be. The number of features that the model chooses at random to decide how to split each node is called mtry. This is another important setting. The trees get more different when the mtry value is low. This can help stop the model from fitting too well. But if mtry is very low, the model might not be as accurate because strong predictors are not always taken into account at splits. When mtry is high, the same strong predictors are used at a lot of splits. This can improve accuracy, but it also makes the trees appear similar, which makes it less useful to combine them. But when the trees are too similar, the ensemble loses its variety. This decreases the effect of reducing variance on Random Forest, which means that the model can start to fit the training data too closely. The minimum node size is another important setting. It determines the minimum amount of observations that must be present in a terminal node or leaf. The trees can grow very deep if this number is small. If only one decision tree is used, this would cause overfitting. But in Random Forest, the averaging across many bootstrapped trees lowers that risk. If it is too big, the trees may not be able to see patterns in the data because they are too shallow. The out-of-bag (OOB) error estimate is another useful part of Random Forest. Since each tree is trained on a bootstrap sample, about a third of the data is not used for that tree. This data can be used as a validation set. The

average prediction error on these left-out samples provides an internal measure of the performance of the model. This means that OOB error is a simple way to adjust hyperparameters without needing a separate validation set. Finally, each tree is trained on a sample of the data that was taken from the original data. Balanced sampling is a type of bootstrap where the minority class is sampled more often than it actually appears in the data. This helps the model learn patterns from classes that are not well represented, but it changes the way the samples are taken, making it not a standard bootstrap anymore.

4.4 Boosting

Boosting is a method that combines weak learners (simple models that perform a bit better than random guessing) to make a strong learner, which makes predictions more accurate. In Random Forest, each model is trained separately and then combined by averaging or voting. In boosting, models are trained one after the other. Every new model learns how to fix the mistakes that the last model made in its predictions. The first step in boosting is to make a simple first model that makes predictions about the training data. Then, the algorithm finds the examples that the first model got wrong or predicted wrong. The next model is taught to find and fix the mistakes that the last model made. This process goes on, and each new model that is added to the ensemble focuses on the mistakes that the previous model did not fix. The final prediction is made by putting all of the models together. A weighted sum is usually used for this, where models that do better get more weight. But boosting is more likely to overfit because each model builds on the one before it. This is especially true when there is a lot of noise in the data or when there are too many models in the ensemble.

XGBoost (Extreme Gradient Boosting) is an efficient machine learning method that builds models in sequence to improve predictions. It includes improvements such as regularization, sparsity-aware algorithms, and optimized computation, which make it faster and more accurate (Chen & Guestrin, 2016). XGBoost uses the same idea of progressive learning as traditional boosting, but it adds L1 and L2 regularization to control model complexity and improve generalization. These are penalties added during the training of the model to prevent overfitting. L1 regularization helps by setting some coefficients to zero. These coefficients are the weights given to features such as words or metadata, so setting them to zero removes predictors that are not useful. L2 regularization, on the other hand, makes all feature weights smaller to avoid extreme values. These penalties stop trees from getting too complicated and

help stop overfitting, especially when there are a lot of features in the data. Regularization reduces model complexity by shrinking or removing less important predictors, which makes more clear which features are more meaningful for the model. This helps to show only the most important features. This is especially helpful for text data with a lot of dimensions, since models can get too complicated very quickly. Also, XGBoost has automatic early stopping features that check how well the model is performing on validation data. When there is no change after a set number of rounds, the model stops training. This feature stops the program from adding models that are not needed, which could cause overfitting. It also helps the model learn patterns from the training data without having to keep training. Most of the time, the tuning of the boosting methods needs to be done manually to pick the right number of boosting iterations, which is how many trees are added to the model during training. XGBoost also works well with sparse data, which is important for text classification tasks that use TF-IDF features and have a lot of zeros in the dataset. The method uses sparse matrix processing, which only works with values that are not zero and skips over those that are. This is very important for TF-IDF features, which often make datasets that are large and sparse. XGBoost can handle high-dimensional text data without using too much memory or taking too long to compute.

There are a lot of hyperparameters in XGBoost that change how the model learns and can keep the model from overfitting. The maximum depth of each weak learner is an important setting because it controls how much detail the model can find. Deep learners can make the model overfit, while shallow learners make more simplistic models that usually work better on a wider range of data. The learning rate shows the model how much each new weak learner will change the model. A lower learning rate usually means that learning takes longer but is more stable. A higher learning rate makes learning faster, but it also makes it more likely that the model will fit the data too closely. Adding some randomness to subsampling parameters like the percentage of training data and the number of features used for each weak learner helps the model work better on new data. Regularization parameters keep the model from getting too complex. L1 regularization works by setting some feature weights to zero, which effectively removes predictors that do not add much value. L2 takes a different approach, shrinking all weights to avoid extreme values and making the model more stable. The minimum child weight controls how many examples must appear in a leaf node, preventing overly specific or shallow splits. When the dataset is imbalanced, adjusting the positive class weight shifts the focus toward underrepresented cases and makes the algorithm

more sensitive to them. Training can also be stopped early if performance on validation data stops improving after a set number of rounds, which avoids unnecessary learners and reduces the risk of overfitting.

4.5 Model Evaluation Metrics

To check how well the models worked, we looked at different evaluation metrics. The results can be put into a confusion matrix, which shows the difference between what was predicted and what was actually true. It has four main parts: true positives, which are the fake or helpful reviews correctly found; false positives, which are reviews marked as fake or helpful but are not; false negatives, which are real fake or helpful reviews that the model missed; and true negatives, which are the reviews correctly marked as real or not helpful. Recall is one key metric since it tells us how many of the real positive cases were identified. This matters because the goal was to catch fake and helpful reviews. When recall is higher, the model manages to catch more of the reviews that matter. Precision looks at something different, telling us how many of the reviews the model marked as positive were truly correct. This stops the model from wrongly flagging too many reviews. Because both precision and recall matter, the study also used the F1-score, a single measure that combines them by taking their mean. This measure is especially useful when we want a balance between catching positive cases and making accurate predictions. We also included accuracy as a general measure of how often the model's predictions are correct overall, although it can be misleading when the classes are imbalanced. Finally, we reported the AUC-ROC, a threshold-independent measure that shows how well the model separates the two classes. However, this measure was not used for comparing models, since it does not reflect trade-offs between different types of errors. Hand (2009) points out that the AUC is not always as reliable as it seems. One reason is that the measure quietly shifts the weight it gives to misclassification costs at different points along the curve. As a result, a model can look impressive in terms of AUC while at the same time failing to reflect the real goals of the analysis. In other words, strong AUC scores do not always mean the model is actually performing in a way that matters for the problem at hand.

5. RESULTS

A key principle of this analysis is to ensure fair evaluation and avoid overfitting. To this end, the dataset is split once into training and test sets, with stratification based on the fake review label. All model development, including cross-validation and hyperparameter tuning, is conducted exclusively on the training set. The test set is kept separate and used once to get an unbiased estimate of how effectively the model performs on new data. By this method we can use the unseen data, and avoid making the models too specific to the training data.

5.1 Hyperparameter Tuning of the Models

Hyperparameters are important for balancing recall, precision, and overall generalization. In this study, tuning focuses on exploring parameter ranges that prevent overfitting and improve model stability. While several evaluation metrics are reported, particular attention in the results is given to the F1-score, since it summarizes the balance between recall and precision in a single value.

For LASSO regression, we use cross-validation to find the value of the penalty parameter lambda that gives us the lowest validation error. The cross-validation curves in Figure 5 show that performance remains relatively stable across a wide range of λ values, with the optimal λ located near the one-standard-error rule in both tasks. The best lambda for finding fake reviews is 0.0025, and the best AUC for cross-validation is 0.697. The best lambda for predicting helpfulness was 0.00013, and the best AUC was 0.722. These curves suggest that the models are not sensitive to small changes in λ, which increases confidence in their robustness. Then, the model is tested on the test set with a set probability threshold of 0.5. There are many ways to measure performance, such as precision, recall, F1-score, accuracy, and AUC. The F1-score is the most important measure for this study, since it shows how well precision and recall work together.

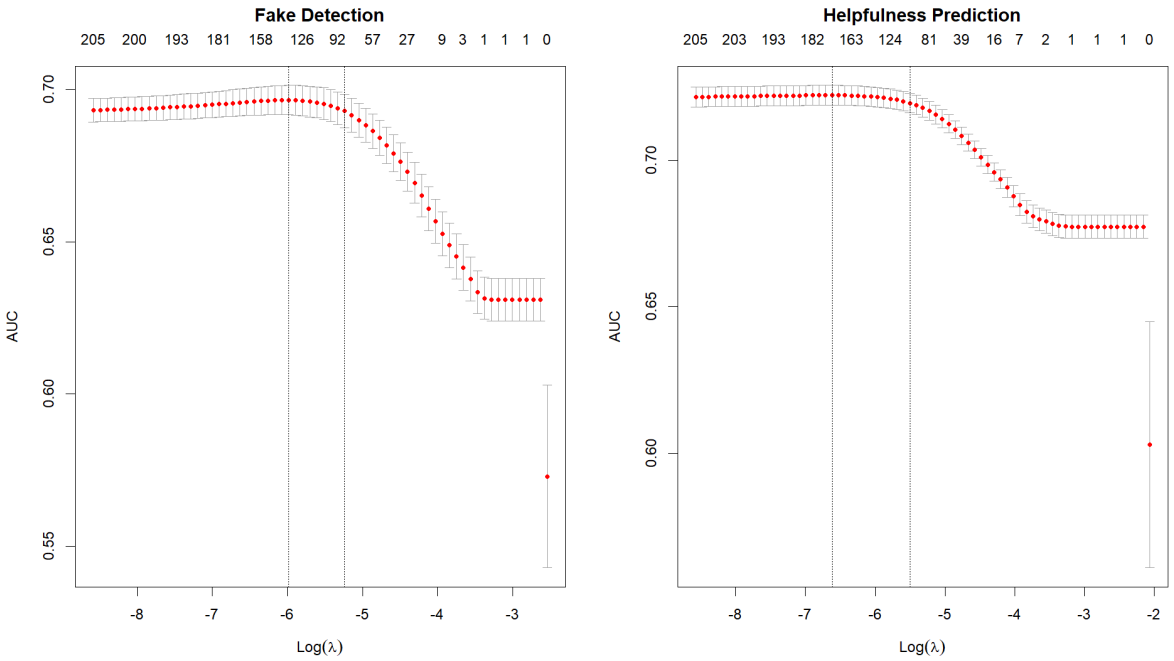

Figure 5: Cross-Validation Error Curves for LASSO

In Random Forest, the main parameter to tune is the number of predictors that are used at each split, called mtry. We test different values, starting from a quarter of the default up to twice the default, where the default is the square root of the total number of predictors. To pick the best value, we use the Out Of Bag (OOB) error, which gives an internal estimate of model performance. As shown in Figure 6, the OOB error curves are relatively flat across the tested values, indicating that performance is not highly sensitive to the exact choice of mtry. For fake review detection, the minimum error is reached at mtry = 10, with an OOB error of 0.161, while for helpfulness prediction the lowest error occurs at mtry = 14 and an OOB error of 0.185. These values mark the minima, but the overall differences across other mtry values are very small and not visually pronounced in the plots, which confirms that the model remains stable across a broad range of mtry values. This approach makes it possible to tune the model without a separate validation set, which is helpful because the data are imbalanced and have many predictors. The number of trees is set at 1,000, which is enough to stabilize the results and reduce variance. We keep the classification threshold at 0.5 for evaluation. In addition to the standard Random Forest, we train a balanced version by applying class weights to correct for the skewed distribution of the target variables. For fake review detection, fake reviews, which is the minority class, is given higher weight so that misclassifying a fake review is penalized more strongly than misclassifying a real one. For

helpfulness prediction, higher weight is assigned to the minority class of helpful reviews. The balanced model uses the same hyperparameters as the standard Random Forest (mtry, number of trees, and classification threshold). Both versions are then compared to see if balancing the classes improved performance.

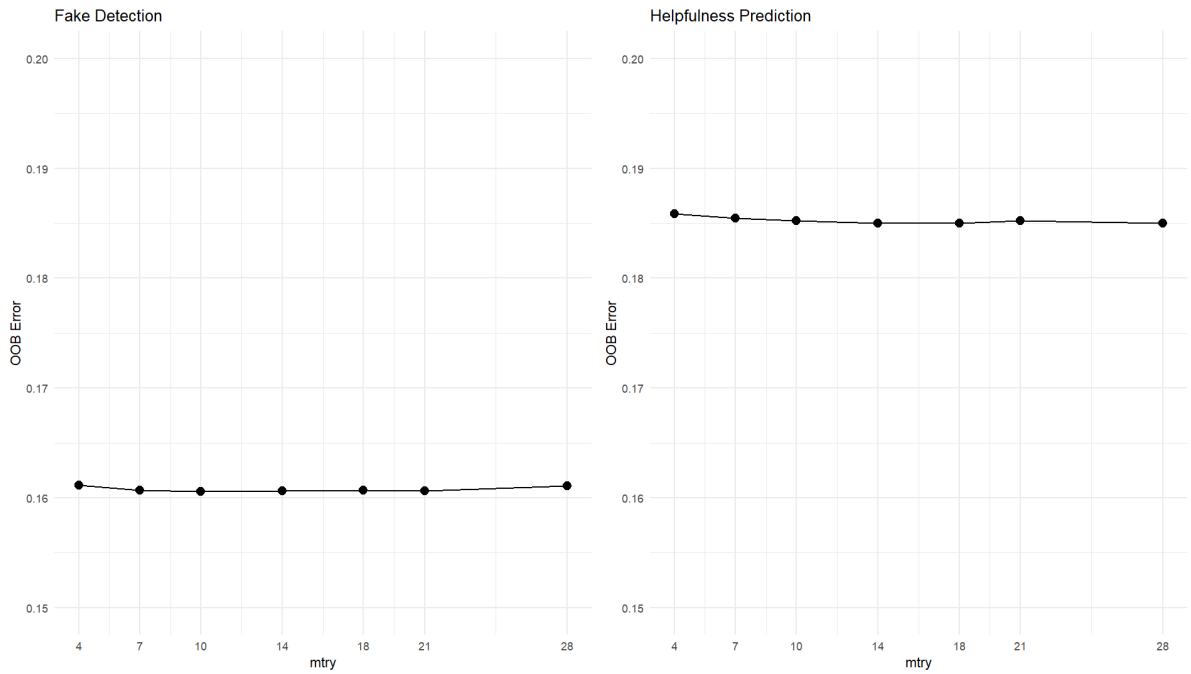

Figure 6: Out Of Bag (OOB) Error Curves for Random Forest

Five hyperparameters are tuned for XGBoost. The learning rate is tested at 0.01, 0.05, and 0.1, so we can see how quickly the model can learn from its mistakes. When the learning rate is lower, the model learns slower, but more steadily. This often means that it needs more trees to have a good performance. The model can learn faster with a higher learning rate, but it also runs the risk of fitting noise in the data. Maximum tree depth is set between 3 and 10, because shallow trees can help the model from overfitting, and deeper trees let the model capture more complex feature interactions. The ratios of rows and predictors that we use in each boosting round range from 0.7 and 1.0. Using 70% of the feature data makes the model less likely to overfit and adds randomness. Using all rows or predictors gives the model the most information. During the search, the number of boosting rounds are tested at 200 and 500 to find a good balance between performance and computation. In the final models, the number of rounds is increased to 1,500, in order to make the boosting process more stable. Hyperparameter selection is guided by cross-validation with AUC as the optimization criterion, and the final test set results are reported at the conventional classification threshold

of 0.5. As can be seen in Figure 7, mean AUC values vary within a very narrow range (about 0.01), and the associated standard deviations (≈0.006-0.011) confirm that differences between configurations are small and not practically meaningful. This indicates that performance is stable across the tested hyperparameter grid. For fake review detection, the highest AUC values appear with shallow trees and a moderate number of boosting rounds. For helpfulness prediction, performance also improves with deeper trees and more rounds, but the improvement slows down after a certain point. The best configuration for fake review detection has a learning rate of 0.05, a maximum depth of 4, subsample ratio of 0.8, a column subsample ratio of 0.8, and 296 boosting rounds, with a cross-validated AUC of 0.721. For helpfulness prediction, the best configuration is the same, with 258 boosting rounds and a cross validated AUC of 0.735.

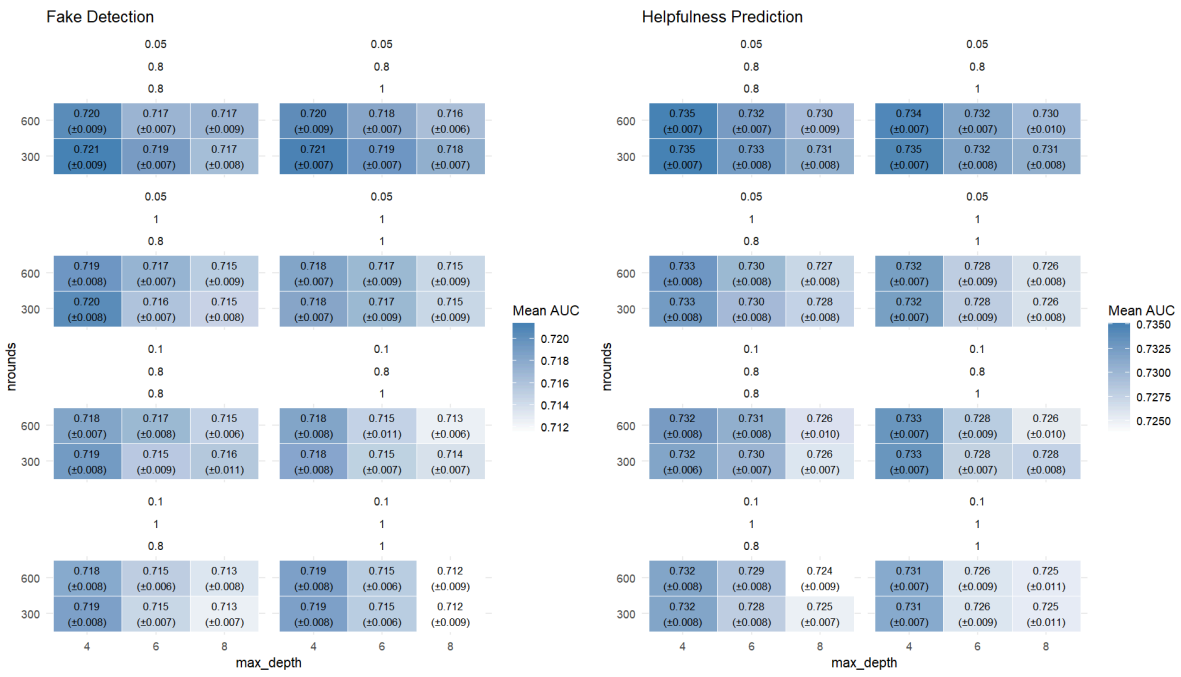

Figure 7: XGBoost Cross-validation AUC Heatmaps

5.2 Model Performance Comparison

Table 2 presents the performance of the three models on the test set for detecting fake reviews. Accuracy is fairly similar across them, landing around 0.77 to 0.78, while the AUC values stay within a narrow band of 0.68 to 0.71. Recall, on the other hand, is really low. Random Forest and Balanced Random Forest reach only 0.06, and XGBoost is just 0.12. In practice, this means that the models catch only a small share of the actual fake reviews.

Precision looks a little better, ranging from 0.53 to 0.60, so when a review is flagged as fake, the models are usually right. Even so, the overall F1-scores remain weak, showing the imbalance between precision and recall. Of the three, XGBoost delivers the best trade-off, with an F1 of 0.198, while LASSO and Random Forest trail at lower levels. Balanced Random Forest was expected to help by adjusting for class imbalance, yet the results tell a different story. Both recall and F1 were almost identical to the standard Random Forest, suggesting that class weighting offered no real advantage in spotting fake reviews.

Algorithm	**Accuracy**	**Precision**	**Recall**	**F1-Score**	**AUC**
LASSO Regression	0.77	0.53	0.08	0.13	0.68
Random Forest	0.78	0.59	0.06	0.10	0.70
Balanced Random Forest	0.78	0.59	0.06	0.10	0.70
XGBoost	0.78	0.60	0.12	0.20	0.71

Table 2: Performance Metrics for Fake Review Detection

In all of the models, performance is better for predicting helpfulness (Table 3). The accuracy is somewhat better than in fake detection, with values between 0.73 (LASSO) and 0.73 (XGBoost). The precision values remain above 0.60, and the recall values are higher than those for fake detection but still low, ranging from 0.24 for Balanced RF to 0.32 for XGBoost. Also, the F1-scores are higher, and XGBoost performs better than the other models again (F1 = 0.42). The AUC values are between 0.71 and 0.74, with XGBoost having the best ability to distinguish the difference between helpful and unhelpful reviews. Balanced Random Forest does not provide any real benefit over the standard version when it comes to fake detection; the results are almost the same for all metrics.

Algorithm	Accuracy	Precision	Recall	F1-Score	AUC
LASSO Regression	0.73	0.61	0.25	0.35	0.73
Random Forest	0.73	0.64	0.24	0.35	0.72
Balanced Random Forest	0.73	0.64	0.24	0.34	0.72
XGBoost	0.73	0.60	0.32	0.42	0.74

Table 3 Performance Metrics for Helpfulness Prediction

These results suggest that XGBoost has the best balance between precision and recall for both tasks, regarding the highest F1-scores. The low recall in fake review detection, though, shows that it is difficult to reliably find deceptive content, even with advanced machine learning techniques. For an overview of the classification outcomes, we include the confusion matrices of each model in the Appendix.

The differences between models are the same during training as they are on the test set. The AUCs for LASSO are 0.70 for fake detection and 0.72 for helpfulness, which is very close to the test AUCs of 0.68 and 0.73. The out-of-bag accuracies of Random Forest are 0.84 for fake and 0.82 for helpfulness, which are also better than the test accuracies of 0.78 and 0.73. This makes sense, since OOB estimates come from resampling the training data. For XGBoost, the cross-validated AUCs reached 0.72 for fake detection and 0.74 for helpfulness, which almost match the test values of 0.71 and 0.74. Seeing training and test numbers line up this closely suggests the models transferred well to new data and that the tuning process kept overfitting under control.

5.3 Feature Importance Analysis

To answer the research question and test the hypotheses, the study also looks at which features are most important for the models. Examining feature importance can help to detect whether fake reviews can be predicted with textual cues, whether helpful reviews are linked

to depth and readability, and if structural aspects play as strong a role as the words themselves.

The LASSO regression provides directly interpretable coefficients, which shows how strongly each predictor is associated with fake and helpful reviews. For fake review detection (Table 4), positive coefficients indicate words that increase the likelihood of a review being classified as fake. Negative coefficients, on the other hand, point to terms that are linked with genuine reviews. Words such as “restaurant”, “asked”, and "told" are positively associated with fake reviews, while more concrete food-related terms like “pork”, “sandwich”, and “sweet" are negatively associated, meaning that these terms indicate authentic reviews. This pattern shows that fake reviews use more general, conversational language than specific product descriptions.

LASSO selects a different set of terms when predicting helpfulness (Table 4). Words like “pretty”, “kind”, and “tough” are linked to reviews that are rated as helpful. Words like “asked”, “staff”, “food”, and “recommend” have negative coefficients. This means that reviews that talk about personal impressions or give descriptive details are more likely to be helpful. On the other hand, vague or repetitive mentions of “food” and “service” are less convincing. Some words, like “came” and “probably”, have the strongest coefficients, but the interpretation of these words is more difficult.

Feature (Fake)	Coefficient	Feature (Helpfulness)	Coefficient
restaurant	3.45	pretty	3.41
asked	3.42	came	2.64
told	3.31	probably	2.43
waiter	2.63	kind	2.10
beef	2.25	though	1.72
pork	-4.76	asked	-5.96
sandwich	-3.16	staff	-5.60
pretty	-2.53	food	-5.03
sweet	-2.39	told	-4.84
cheese	-2.21	recommend	-4.47

Table 4: Top LASSO Coefficients for Both Predictions

The Random Forest models show a different way that features are important than LASSO. Random Forest gives permutation importance scores instead of regression coefficients that show the direction of association. These scores show how much predictive accuracy decreases when a feature is randomly permuted. The model relies more on that feature when the importance scores were higher, but the scores do not show if the effect is positive or negative. Figure 8 shows that structural and stylistic features are the most important for finding fake reviews. Rating is the best predictor, followed by review depth, subjectivity, and readability. The model often uses extreme ratings, short reviews, and language that is either simpler or more opinionated to find the difference between fake and real reviews. Lexical

features such as “food”, “service”, “restaurant”, and “staff” are also important, but they do not contribute as much as the text's overall structure and tone.

Figure 8 also shows that for helpfulness prediction, review depth is by far the most important feature, with rating and readability again close behind. This shows that reviews that are longer, clearer, and written in a more readable style are identified as helpful most of the time. Lexical cues like “great”, “pretty”, “like”, and “service” are also in the top ranks, but their role is secondary to structural features. Taken as a whole, the results suggest that single words do carry some signal, but Random Forest leans much more on the style and structure of the review than on the specific vocabulary.

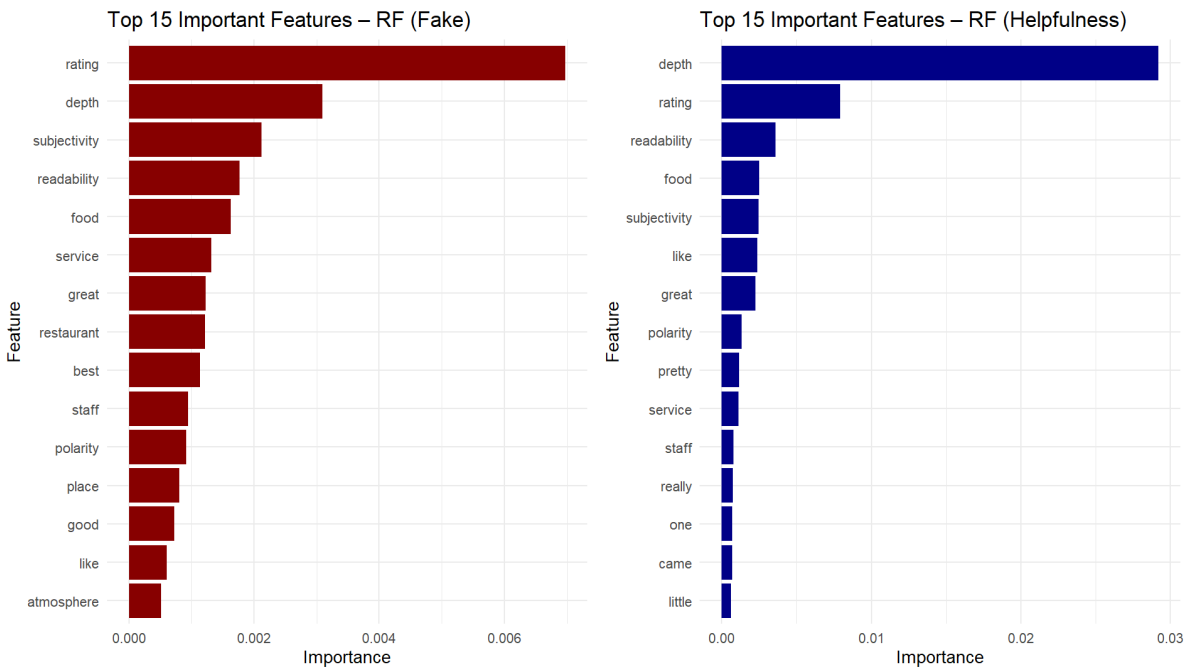

Figure 8: Top 15 Random Forest Features for Both Predictions

The XGBoost models highlighted yet another set of important features, with importance measured by gain, which reflects how much each feature contributed to reducing error across all decision trees. Unlike LASSO, where coefficients directly indicate direction and magnitude, XGBoost provides relative importance scores that show how influential each feature was in the model’s decision-making.

As can be seen in Figure 9, for fake review detection, the most important features are rating, depth, and polarity, alongside several text-based terms such as “restaurant”, “great”, and “food”. Reviews that have higher ratings and fewer words (low depth) are more likely to be fake. Polarity shows that the sentiment in the review also helps to distinguish between fake and real reviews. The word-based features show that fake reviews often use general and positive words (like “great”, “restaurant”, and “food”). They also show that fake reviews may use more general words than specific product references.

When it comes to predicting helpfulness (Figure 9), review depth is the strongest feature, with rating and subjectivity coming next. Reviews that are longer and give more detail tend to be judged helpful, while short ones usually are not. Certain words such as “food,” “great,” “staff,” and “pretty” also show up among the top predictors, which suggests that both detail and subjective impressions are important in how helpful a review seems. Readability and polarity matter less, pointing to the idea that length and specificity carry more weight than sentiment alone.

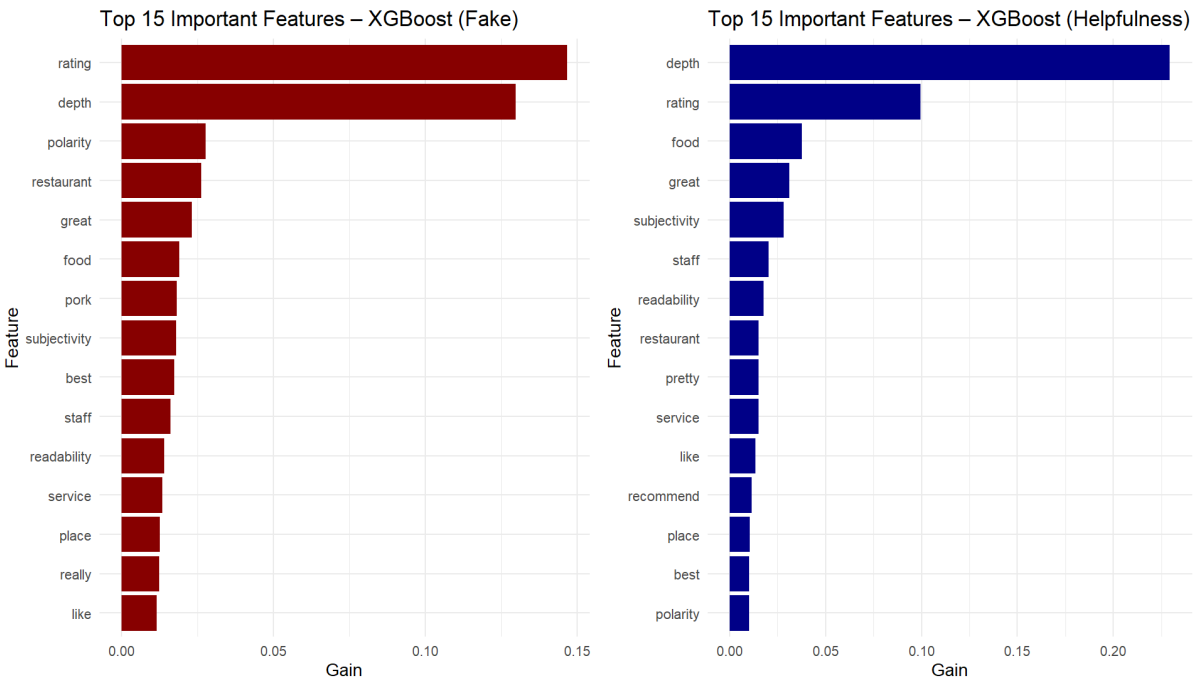

Figure 9: Top 15 XGBoost Features for Both Predictions

Looking at the feature importance results show that words matter, but structure and style are also equally important in the interpretation of the reviews. This is especially relevant for the hypotheses. H1 stated that helpful reviews with more depth and readability are more likely to be rated as helpful whether they are fake or real. This hypothesis is strongly supported because review length consistently ranked as the most influential predictor of helpfulness across all models, with readability also contributing, though to a lesser extent. H2 stated that fake reviews are considered more helpful than real reviews. This hypothesis is not supported because fake reviews did not receive higher helpfulness scores than real ones, and the main drivers of helpfulness applied equally to both groups. H3 stated that easy to read text and detailed content best explain why both fake and real reviews are rated as helpful. This hypothesis is also supported because readability and depth clearly dominate over individual words in predicting helpfulness. Furthermore, when combining these results, the broader research question which asked whether textual and structural features can explain differences in fake review detection and perceived helpfulness is answered affirmatively. The models indicate that although certain words are influential, the structure and style of reviews serve as the most significant indicators, especially for helpfulness. This shows that it is possible to find fake reviews based on the text, but that the perception of helpfulness is mostly based on how deep and readable the text is, no matter how real it is.

6. CONCLUSION & DISCUSSION

This thesis tries to answer the research question: How do the textual features of fake reviews (polarity, subjectivity, readability, depth) differ from authentic reviews, and which features best explain their perceived helpfulness? To address this, three machine learning methods are applied to a dataset of Yelp restaurant reviews: LASSO, Random Forest, and XGBoost. The results provide three main insights. First, predicting helpfulness is more straightforward than detecting deception. Reviews that are longer, more detailed, and easier to read are judged as more helpful across all models, regardless of whether they were fake or real. Second, the detection of fake reviews remains challenging, because deceptive reviews closely mimic real ones. Even advanced models like XGBoost have low recall, which underlines the difficulty of separating authentic from deceptive content. Third, the type of features matters. LASSO highlights individual words with strong associations, but Random Forest and XGBoost

emphasize structural and stylistic features, with depth, readability, and rating being the most reliable indicators.

These findings have implications for both research and practice. For academics, the results contribute to the literature by showing that structural and stylistic elements in the reviews explain helpfulness better than lexical features alone. They also show that detecting deception is more challenging because fake and real reviews share many similarities. From a managerial perspective, the insights suggest two key takeaways. The usefulness of the review sections can be improved when promoting reviews that are longer, clearer, and more detailed, in order to enhance consumer trust. In addition, reviews that are easy to read and detailed are more likely to support customer decision making and also strengthen the credibility of the platform.

There are also some limitations to this study. One of them is that only Yelp restaurant reviews are used, which makes it hard to say whether the same patterns hold true in areas like retail, hotels, or travel. The features that the models include are also restricted. They include only the text of review and some basic structural elements. Missing are signals about the reviewers themselves, such as their posting history or credibility, and also contextual details like when the review is written. These kinds of features might have improved performance. Another point is that many of the features capture surface-level traits instead of deeper meaning or context. This may explain why the models struggle with fake detection, since shallow signals are rarely enough to expose deception. Expanding to richer datasets and including features such as semantic embeddings or metadata could help future research go further. Overall, the findings point in two directions. Reviews that are clear, detailed and easy to read tend to be seen as helpful. Detecting fakes, in contrast, is still difficult, because deceptive writing blends too well with real writing.

7. REFERENCES

Ahmad, W., & Sun, J. (2018). Modeling consumer distrust of online hotel reviews. *International Journal of Hospitality Management*, *71*, 77–90. https://doi.org/10.1016/j.ijhm.2017.12.005

Banerjee, S., Chua, A. Y. K., & Kim, J. (2017). Don't be deceived: Using linguistic analysis to learn how to discern online review authenticity. *Journal of the Association for Information Science and Technology*, *68*(6), 1525–1538. https://doi.org/10.1002/asi.23784

Barbado, R., Araque, O., & Iglesias, C. A. (2019). A framework for fake review detection in online consumer electronics retailers. *Information Processing & Management*, *56*(4), 1234–1244. https://doi.org/10.1016/j.ipm.2019.03.002

Cao, Q., Duan, W., & Gan, Q. (2011). Exploring determinants of voting for the "helpfulness" of online user reviews: A text mining approach. *Decision Support Systems*, *50*(2), 511–521. https://doi.org/10.1016/j.dss.2010.11.009

Chen, Y., & Xie, J. (2008). Online Consumer Review: Word-of-Mouth as a New Element of Marketing Communication Mix. *Management Science*, *54*(3), 477–491. https://doi.org/10.1287/mnsc.1070.0810

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, *13-17-August-2016*, 785–794. https://doi.org/10.1145/2939672.2939785

Chua, A. Y. K., & Banerjee, S. (2015). Understanding review helpfulness as a function of reviewer reputation, review rating, and review depth. *Journal of the Association for Information Science and Technology*, *66*(2), 354–362. https://doi.org/10.1002/asi.23180

Chua, A. Y. K., & Banerjee, S. (2016). Helpfulness of user-generated reviews as a function of review sentiment, product type and information quality. *Computers in Human Behavior*, *54*, 547–554. https://doi.org/10.1016/j.chb.2015.08.057

D. Bates. (2013). Samsung ordered to pay $340,000 after it paid people to write negative online reviews about HTC phones. *DailyMail*. https://www.dailymail.co.uk/sciencetech/article-2476630/Samsung-ordered-pay-340-000-paid-people-write-negative-online-reviews-HTC-phones.html

Dagly, D. (2020). Fake review detection dataset. Github. https://github.com/darshandagly/Fake-Review-Detection

Ghose, A., & Ipeirotis, P. G. (2011). Estimating the Helpfulness and Economic Impact of Product Reviews: Mining Text and Reviewer Characteristics. *IEEE Transactions on Knowledge and Data Engineering*, *23*(10), 1498–1512. https://doi.org/10.1109/TKDE.2010.188

Hand, D. J. (2009). Measuring classifier performance: A coherent alternative to the area under the ROC curve. *Machine Learning*, *77*(1), 103–123. https://doi.org/10.1007/s10994-009-5119-5

Heydari, A., Tavakoli, M. ali, Salim, N., & Heydari, Z. (2015). Detection of review spam: A survey. *Expert Systems with Applications*, *42*(7), 3634–3642. https://doi.org/10.1016/j.eswa.2014.12.029

Kohli, R., Devaraj, S., & Mahmood, M. A. (2004). Understanding determinants of online consumer satisfaction: A decision process perspective. *Journal of Management Information Systems*, *21*(1), 115–136. https://doi.org/10.1080/07421222.2004.11045796

Kuan, K. K. Y., Hui, K. L., Prasarnphanich, P., & Lai, H. Y. (2015). What makes a review voted? An empirical investigation of review voting in online review systems. *Journal of the Association for Information Systems*, *16*(1), 48–71. https://doi.org/10.17705/1jais.00386

Li, M., Huang, L., Tan, C.-H., & Wei, K.-K. (2013). Helpfulness of Online Product Reviews as Seen by Consumers: Source and Content Features. *International Journal of Electronic Commerce*, *17*(4), 101–136. https://doi.org/10.2753/JEC1086-4415170404

Luca, M., & Zervas, G. (2016). Fake It Till You Make It: Reputation, Competition, and Yelp Review Fraud. *Management Science*, *62*(12), 3412–3427. https://doi.org/10.1287/mnsc.2015.2304

Mathwick, C., & Mosteller, J. (2017). Online Reviewer Engagement. *Journal of Service Research*, *20*(2), 204–218. https://doi.org/10.1177/1094670516682088

Mayzlin, D., Dover, Y., & Chevalier, J. (2014). Promotional Reviews: An Empirical Investigation of Online Review Manipulation. *American Economic Review*, *104*(8), 2421–2455. https://doi.org/10.1257/aer.104.8.2421

Mudambi, & Schuff. (2010). What Makes a Helpful Online Review? A Study of Customer Reviews on Amazon.com. *MIS Quarterly*, *34*(1), 185. https://doi.org/10.2307/20721420

Mukherjee, A., Venkataraman, V., Liu, B., & Glance, N. (2013). *What Yelp Fake Review Filter Might Be Doing?* www.aaai.org

Munzel, A. (2016). Assisting consumers in detecting fake reviews: The role of identity information disclosure and consensus. *Journal of Retailing and Consumer Services*, *32*, 96–108. https://doi.org/10.1016/j.jretconser.2016.06.002

Otterbacher J. (2009). *Proceedings of the 27th International Conference on Human Factors in Computing Systems : April 4-9, 2009, Boston, MA, USA*. ACM Press.

Rinker, T.W. (2019). lexicon: Lexicon Data (Version 1.2.1) [R package]. Retrieved from https://cran.r-project.org/package=lexicon

Rinker, T.W. (2021). sentimentr: Calculate Text Polarity Sentiment (Version 2.9.0) [R package]. Comprehensive R Archive Network (CRAN). https://doi.org/10.32614/CRAN.package.sentimentr

Pang, B., Lee, L., & Vaithyanathan, S. (2002). *Thumbs up? Sentiment Classification using Machine Learning Techniques*. http://reviews.imdb.com/Reviews/

Plotkina, D., Munzel, A., & Pallud, J. (2018). Illusions of truth—Experimental insights into human and algorithmic detections of fake online reviews. *Journal of Business Research*, *109*, 511–523. https://doi.org/10.1016/j.jbusres.2018.12.009

Qazi, A., Shah Syed, K. B., Raj, R. G., Cambria, E., Tahir, M., & Alghazzawi, D. (2016). A concept-level approach to the analysis of online review helpfulness. *Computers in Human Behavior*, *58*, 75–81. https://doi.org/10.1016/j.chb.2015.12.028

Salminen, J., Kandpal, C., Kamel, A. M., Jung, S., & Jansen, B. J. (2022). Creating and detecting fake reviews of online products. *Journal of Retailing and Consumer Services*, *64*, 102771. https://doi.org/10.1016/j.jretconser.2021.102771

Sandulescu, V., & Ester, M. (2015). Detecting Singleton Review Spammers Using Semantic Similarity. *Proceedings of the 24th International Conference on World Wide Web*, 971–976. https://doi.org/10.1145/2740908.2742570

Smith A., & Anderson M. (2016). Online Shopping and E-Commerce, Online Reviews. *Pew Research Center*.

Sudhakaran P., Hariharan S., & Lu S. (2016). A framework investigating the online user reviews to measure the biasness for sentiment analysis. *Asian Journal of Information Technology*, 1890–1898.

The Telegraph Reporter. (2025). Amazon to ban fake reviewers and punish sellers. . *The Telegraph*.
https://www.telegraph.co.uk/business/2025/06/06/amazon-to-ban-fake-reviewers-and-punish-sellers/

Wang, Y., Zamudio, C., & Jewell, R. D. (2023). The more they know: Using transparent online communication to combat fake online reviews. *Business Horizons*, *66*(6), 753–764. https://doi.org/10.1016/j.bushor.2023.03.004

Yao, Y., Viswanath, B., Cryan, J., Zheng, H., & Zhao, B. Y. (2017). Automated Crowdturfing Attacks and Defenses in Online Review Systems. *Proceedings of the 2017 ACM SIGSAC*

Conference on Computer and Communications Security, 1143–1158. https://doi.org/10.1145/3133956.3133990

8. APPENDIX

Variable	Variable	Variable	Variable
fake_binary	helpful_binary	polarity	subjectivity
readability	depth_wordcount	rating	tfidf_great
tfidf_pizza	tfidf_food	tfidf_good	tfidf_place
tfidf_love	tfidf_service	tfidf_best	tfidf_really
tfidf_like	tfidf_chicago	tfidf_get	tfidf_delicious
tfidf_back	tfidf_one	tfidf_time	tfidf_amazing
tfidf_wait	tfidf_restaurant	tfidf_always	tfidf_nice
tfidf_try	tfidf_atmosphere	tfidf_menu	tfidf_beer
tfidf_also	tfidf_beer	tfidf_awesome	tfidf_well
tfidf_definitely	tfidf_little	tfidf_favorite	tfidf_went
tfidf_excellent	tfidf_better	tfidf_staff	tfidf_dinner
tfidf_night	tfidf_ever	tfidf_even	tfidf_eat
tfidf_worth	tfidf_experience	tfidf_salad	tfidf_pretty
tfidf_cheese	tfidf_ordered	tfidf_much	tfidf_everything
tfidf_wine	tfidf_never	tfidf_friendly	tfidf_order
tfidf_got	tfidf_table	tfidf_chicken	tfidf_dish

tfidf_say	tfidf_people	tfidf_can	tfidf_fresh
tfidf_think	tfidf_make	tfidf_going	tfidf_meal
tfidf_first	tfidf_recommend	tfidf_meat	tfidf_times
tfidf_lunch	tfidf_come	tfidf_drinks	tfidf_bit
tfidf_came	tfidf_friends	tfidf_every	tfidf_brunch
tfidf_know	tfidf_tasty	tfidf_fantastic	tfidf_steak
tfidf_loved	tfidf_still	tfidf_bad	tfidf_though
tfidf_hot	tfidf_want	tfidf_price	tfidf_city
tfidf_way	tfidf_take	tfidf_prices	tfidf_sure
tfidf_sandwich	tfidf_two	tfidf_pork	tfidf_last
tfidf_selection	tfidf_day	tfidf_long	tfidf_made
tfidf_sauce	tfidf_tried	tfidf_wonderful	tfidf_dishes
tfidf_right	tfidf_stars	tfidf_many	tfidf_fun
tfidf_perfect	tfidf_big	tfidf_special	tfidf_friend
tfidf_something	tfidf_thing	tfidf_soup	tfidf_small
tfidf_around	tfidf_give	tfidf_said	tfidf_huge
tfidf_next	tfidf_places	tfidf_enough	tfidf_enough
tfidf_must	tfidf_new	tfidf_taste	tfidf_area

tfidf_super	tfidf_dessert	tfidf_server	tfidf_side
tfidf_lot	tfidf_minutes	tfidf_nothing	tfidf_top
tfidf_restaurants	tfidf_dining	tfidf_feel	tfidf_spot
tfidf_waiter	tfidf_bread	tfidf_busy	tfidf_quality
tfidf_years	tfidf_sweet	tfidf_things	tfidf_enjoyed
tfidf_however	tfidf_everyone	tfidf_another	tfidf_took
tfidf_eating	tfidf_different	tfidf_chocolate	tfidf_probably
tfidf_decor	tfidf_quite	tfidf_thought	tfidf_visit
tfidf_overall	tfidf_beef	tfidf_drink	tfidf_find
tfidf_group	tfidf_hour	tfidf_else	tfidf_enjoy
tfidf_large	tfidf_see	tfidf_kind	tfidf_fried
tfidf_especially	tfidf_full	tfidf_room	tfidf_seated
tfidf_home	tfidf_need	tfidf_maybe	tfidf_waitress
tfidf_home	tfidf_list	tfidf_course	tfidf_looking
tfidf_check	tfidf_anything	tfidf_high	tfidf_actually
tfidf_flavor	tfidf_half	tfidf_served	tfidf_without
tfidf_least	tfidf_wanted	tfidf_plate	tfidf_left
tfidf_tables	tfidf_waiting	tfidf_now	tfidf_three

tfidf_couple	tfidf_almost	tfidf_asked	tfidf_let
tfidf_told			

Table 5: Variables Used in the Models

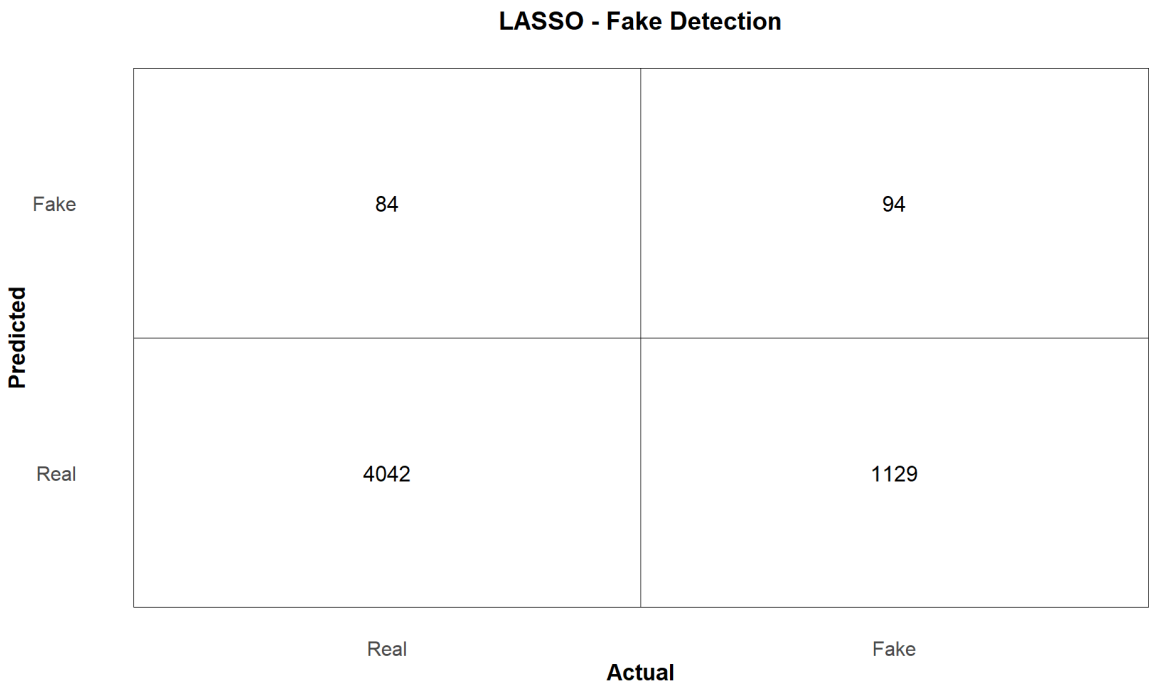

Figure 10: Confusion Matrix of LASSO model for Fake Detection

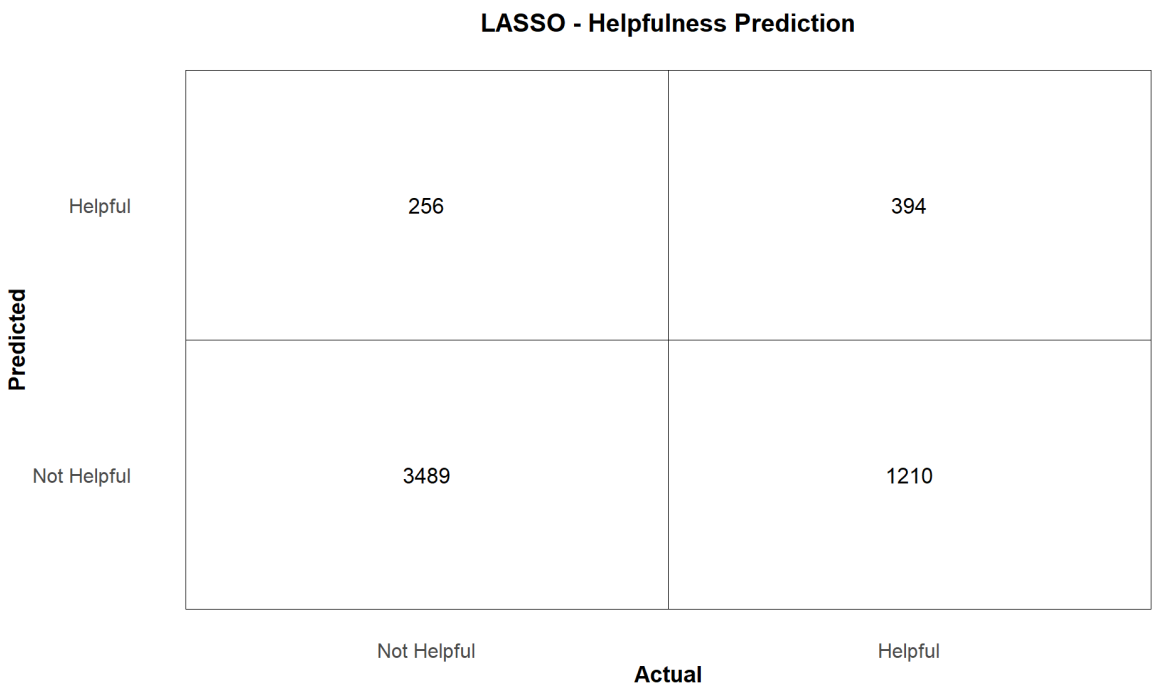

Figure 11: Confusion Matrix of LASSO model for Helpfulness Prediction

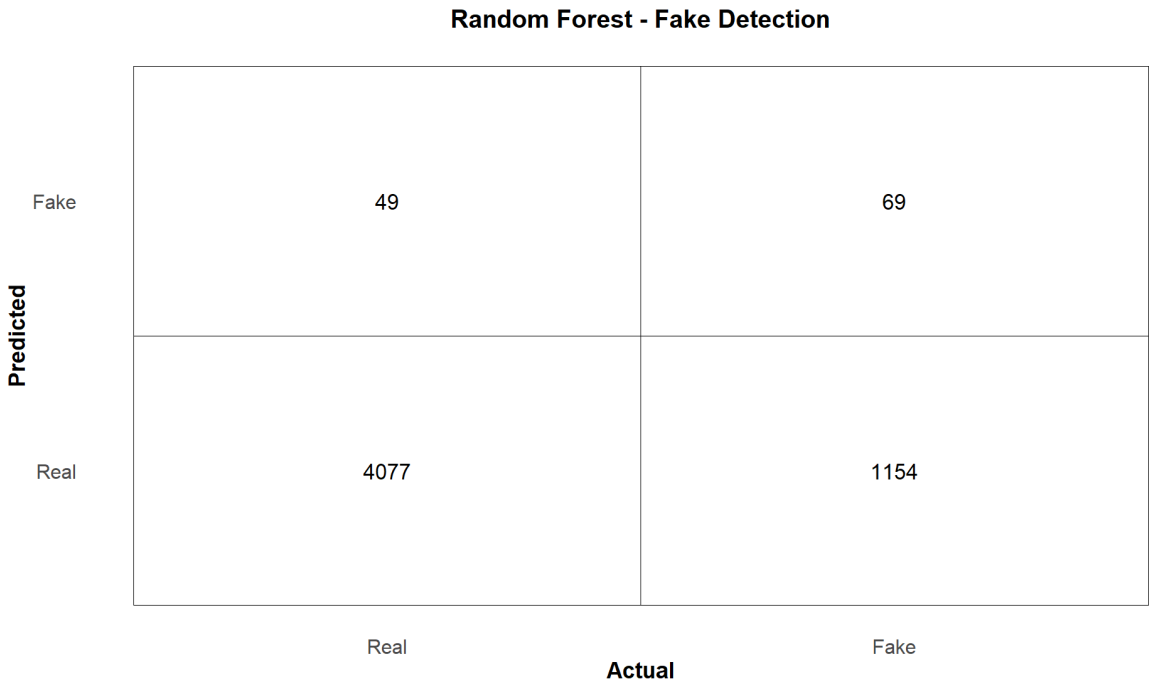

Figure 12: Confusion Matrix of Random Forest model for Fake Detection

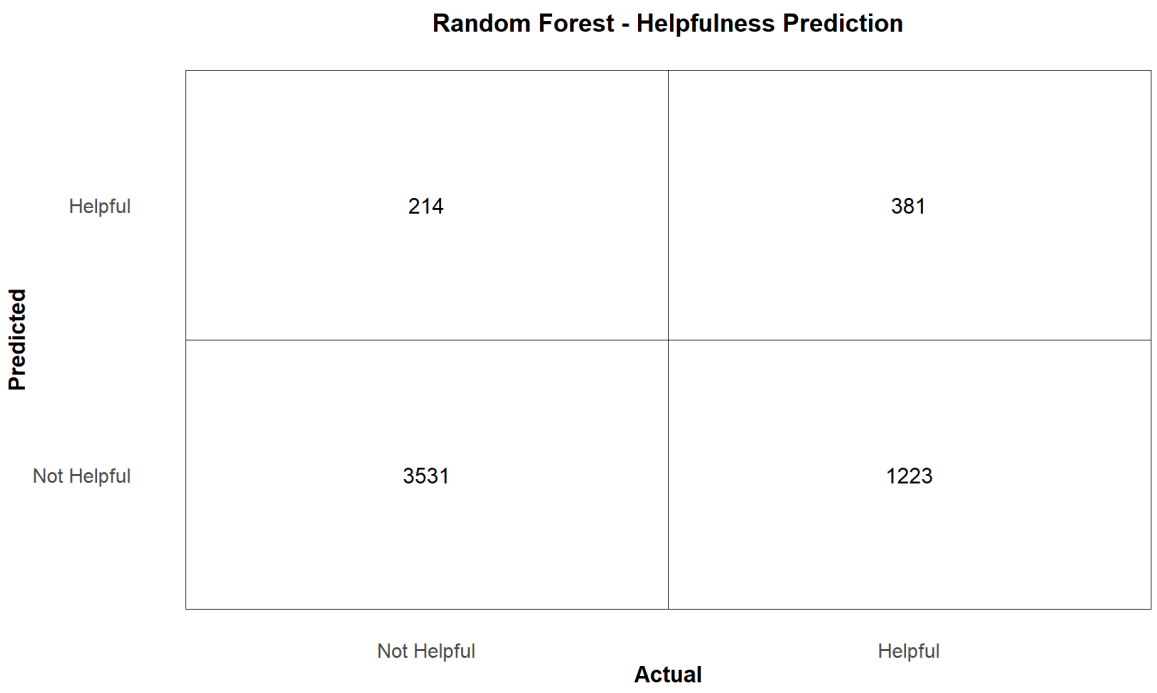

Figure 13: Confusion Matrix of Random Forest model for Helpfulness Prediction

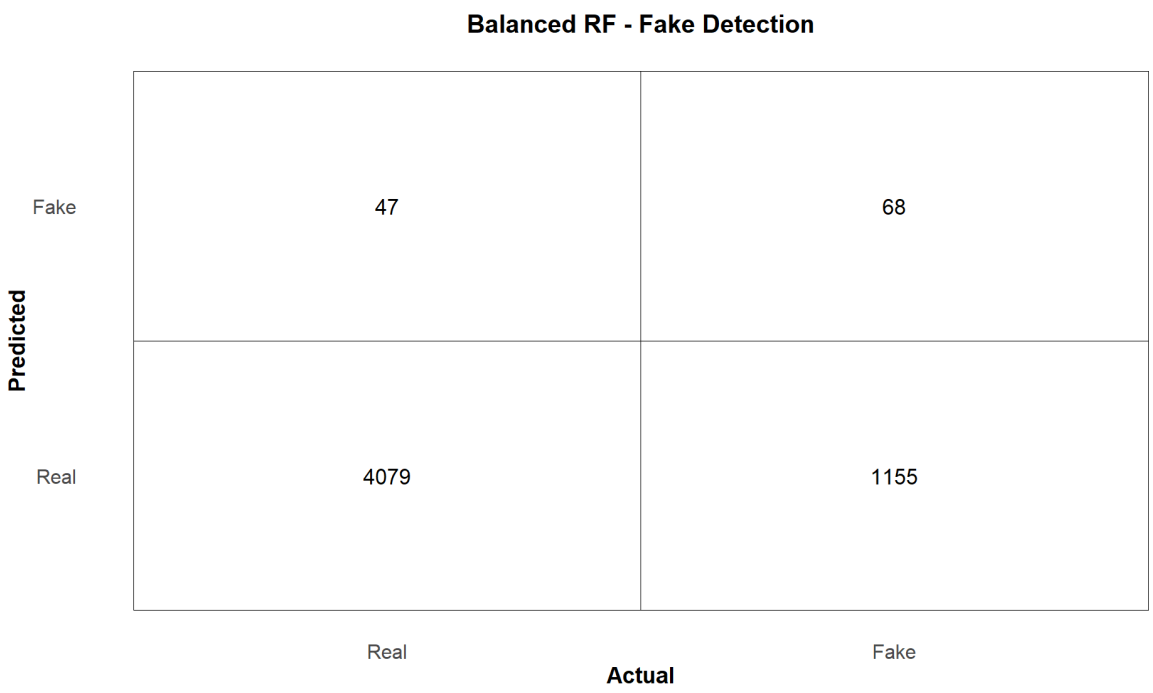

Figure 14: Confusion Matrix of Balanced Random Forest model for Fake Detection

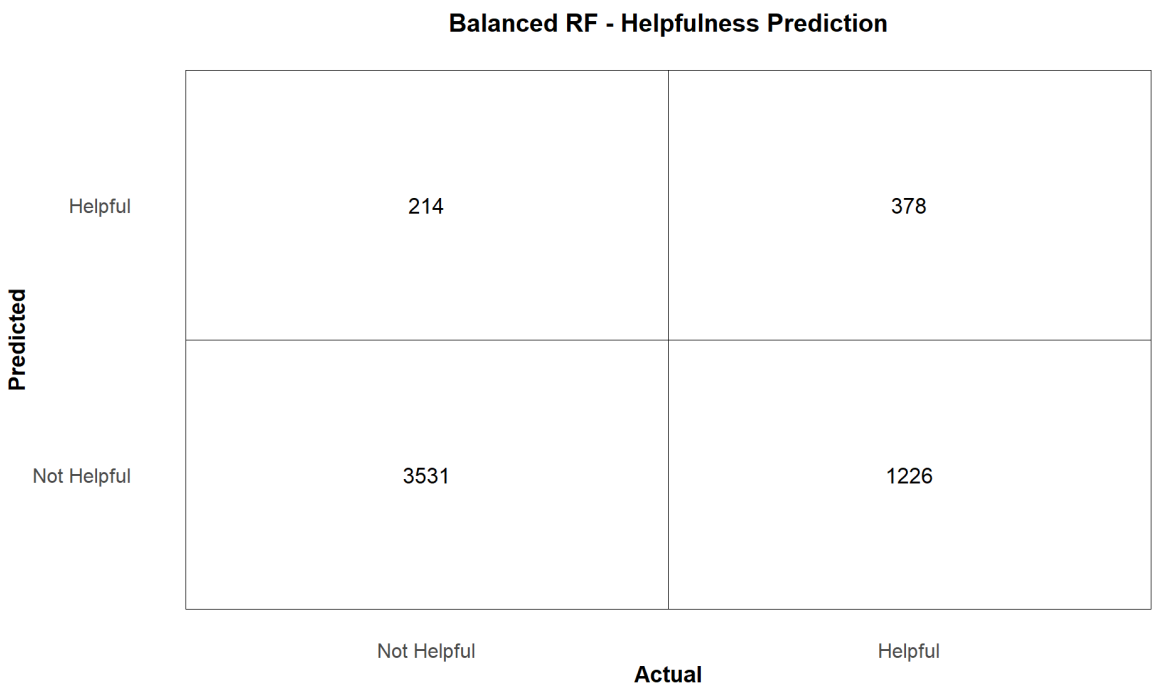

Figure 15: Confusion Matrix of Balanced Random Forest model for Helpfulness Prediction

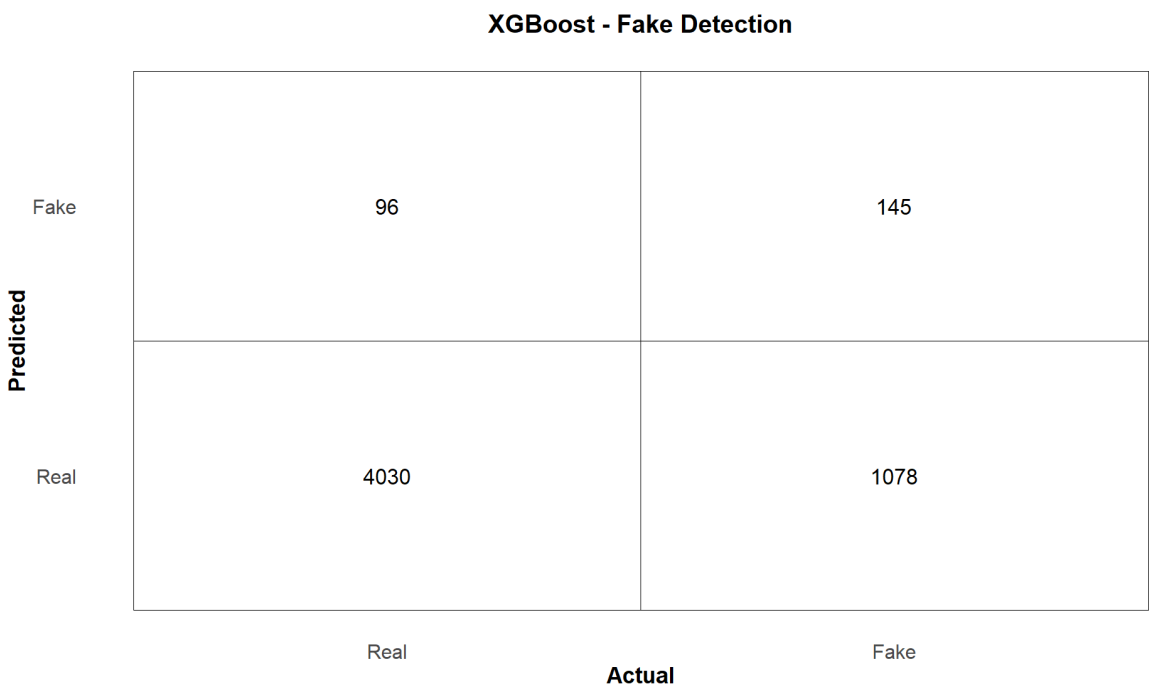

Figure 16: Confusion Matrix of XGBoost model for Fake Detection

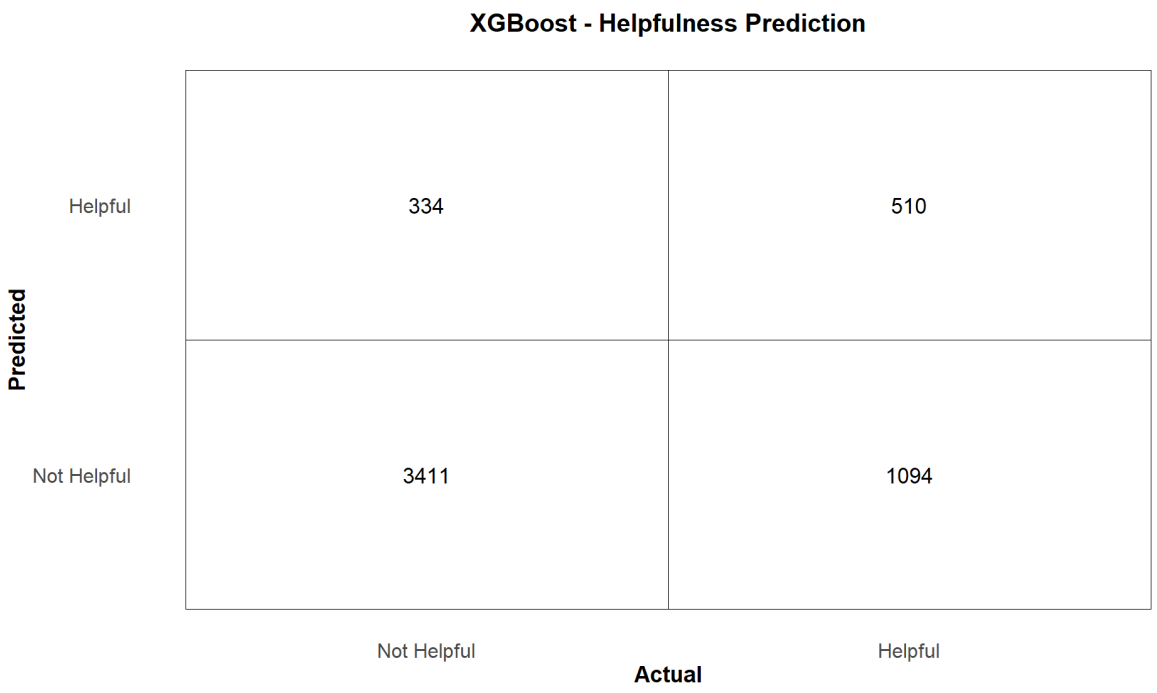

Figure 17: Confusion Matrix of XGBoost model for Helpfulness Prediction